

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Membre de Honoris United Universities
EMSI Casablana / Rabat / Marrakech

RAPPORT DE PROJET DE FIN D'ANNÉE (PFA)

Sujet :

Implémentation et Évaluation d'un Système de Consensus Byzantin
Résilient (PBFT) avec Sécurisation Cryptographique

Réalisé par :

Yassir KADOUARI
Matine ELKASBIJI
Marouane ISMAILI

Filière :

Ingénierie Informatique

Option :

Systèmes & Réseaux / Génie
Logiciel

Encadré par : *M. l'Enseignant de Systèmes Distribués*

Année Universitaire : 2025 / 2026

Table des matières

Remerciements	2
Introduction Générale	3
1 Chapitre 1 : Contexte Général et Problématique	3
1.1 Présentation du Projet	3
1.2 La Problématique et l'Objectif	3
2 Chapitre 2 : Étude Technique et Conception	3
2.1 Le Protocole de Consensus PBFT	3
2.2 Sécurisation Cryptographique	4
3 Chapitre 3 : Réalisation, Expérimentation et Évaluation	4
3.1 Architecture Logicielle de notre Système	4
3.2 Simulations d'Attaques et Résultats Réels	4
Conclusion Générale	4

Remerciements

Nous tenons à exprimer nos sincères remerciements à l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour nous avoir offert le cadre idéal pour la réalisation de ce projet. Nous remercions également notre encadrant académique pour ses conseils précieux, ses critiques constructives et son accompagnement scientifique tout au long du développement de ce Projet de Fin d'Année. Enfin, nous remercions tous les membres du corps professoral de l'EMSI pour la qualité des cours dispensés.

Introduction Générale

Dans les infrastructures informatiques modernes, notamment les réseaux bancaires, industriels et blockchains, la tolérance aux pannes et aux attaques est une préoccupation majeure. Un simple piratage ou une défaillance réseau sur un seul serveur ne doit pas corrompre l'ensemble des données. C'est dans ce cadre que s'inscrit le consensus byzantin, capable de maintenir l'accord entre serveurs sains même en présence de nœuds malveillants (byzantins).

L'objectif de notre Projet de Fin d'Année (PFA) est de concevoir, d'implémenter et d'évaluer un cluster distribué basé sur le protocole **PBFT** (Practical Byzantine Fault Tolerance) en Java 25. Ce rapport détaille notre démarche, allant des fondements mathématiques de la cryptographie utilisée (ECDSA, Shamir) jusqu'à la mise en place d'un dashboard de monitoring interactif et les simulations d'attaques.

1 Chapitre 1 : Contexte Général et Problématique

1.1 Présentation du Projet

Ce PFA vise à développer un protocole de consensus robuste au sein d'un cluster à 4 nœuds ($N = 4$). D'après les théorèmes fondamentaux de tolérance aux pannes byzantines, la borne inférieure de nœuds sains pour résister à f nœuds byzantins est formulée par :

$$N \geq 3f + 1 \quad (1)$$

Pour notre projet, le cluster est configuré pour supporter $f = 1$ panne complète ou attaque malveillante simultanée.

1.2 La Problématique et l'Objectif

Les protocoles de consensus classiques (comme Raft ou Paxos) sont dits "Crash Fault Tolerant" (CFT) ; ils tolèrent les pannes de courant ou d'extinction de serveurs mais échouent dès qu'un nœud envoie volontairement de fausses informations (mensonge). Notre objectif est donc de réaliser une implémentation PBFT garantissant :

- **La Sûreté (Safety)** : L'assurance que deux nœuds sains ne prendront jamais de décisions contradictoires (évitant les bifurcations d'état/forks).
- **La Vivacité (Liveness)** : La certitude que le système continue à enregistrer et traiter des transactions de façon autonome même si un nœud refuse de répondre.

2 Chapitre 2 : Étude Technique et Conception

2.1 Le Protocole de Consensus PBFT

Notre moteur de consensus PBFT ('PBFTEngine.java') orchestre le cycle d'accord en trois phases successives :

1. **Pre-Prepare** : Le Leader (déterminé par la rotation $Leader = View \pmod{N}$) reçoit la transaction client, lui attribue un numéro de séquence, la signe numériquement via ECDSA et la diffuse à tous les pairs.
2. **Prepare** : Chaque réplica vérifie la signature et émet un vote 'PREPARE' à tous les pairs. Dès qu'un nœud possède $2f$ prepares cohérents, il passe à l'état *Prepared*.
3. **Commit** : Dès qu'un nœud est préparé, il émet un message 'COMMIT'. Une fois qu'il a accumulé $2f + 1$ commits de pairs différents, le consensus est validé. La transaction est alors exécutée localement et enregistrée dans les métriques.

2.2 Sécurisation Cryptographique

Notre étude intègre deux piliers cryptographiques majeurs :

- **Signatures ECDSA (secp256r1)** : Chaque message circulant sur le réseau TCP est signé à l'aide de la clé privée ECDSA du nœud émetteur. Cela interdit l'usurpation d'identité et garantit l'authenticité de chaque vote.
- **Partage de Secret de Shamir ($t = 2, n = 4$)** : Pour sécuriser les clés de signature de groupe (TSS), nous avons implémenté le partage polynomial de Shamir modulo le nombre premier $P = 2^{256} - 2^{32} - 977$.

3 Chapitre 3 : Réalisation, Expérimentation et Évaluation

3.1 Architecture Logicielle de notre Système

Le projet a été développé de manière modulaire :

- **NetworkManager** : Moteur de sockets TCP multi-threadé gérant le réseau en maillage complet et l'échange de clés publiques initiales.
- **ByzantineBehavior** : Interface d'injection de fautes en temps réel (**SilentNode**, **EquivocationNode**, **ReplayNode**).
- **MetricsServer** : API HTTP REST intégrée fournissant les données JSON au dashboard.
- **Dashboard Interactif** : Interface moderne en HTML/CSS/JS servie par Nginx permettant de suivre en direct l'état des nœuds, le graphe réseau et les anomalies.

3.2 Simulations d'Attaques et Résultats Réels

Dans notre scénario d'évaluation, nous avons configuré le **Nœud 3 en mode Byzantin Silencieux (silent)**. Nos observations en direct confirment le succès de notre implémentation :

- Le Nœud 3 reçoit les requêtes de consensus mais détruit systématiquement ses Prepare et Commit sortants, simulant une coupure réseau franche.
- Les nœuds sains 0, 1 et 2 continuent à communiquer. Ils accumulent le quorum de Prepare (2/2 reçus) et de Commit (3/3 reçus), permettant au cluster de valider et d'exécuter les transactions sans aucune panne.
- Notre cluster a complété avec succès **145 rounds de consensus** sans la moindre désynchronisation, avec une latence moyenne de consensus comprise entre **10 ms et 25 ms** par transaction.

Conclusion Générale

Ce Projet de Fin d'Année au sein de l'EMSI nous a permis de mettre en pratique et d'approfondir nos compétences en développement de systèmes distribués sécurisés en Java. En concevant ce cluster PBFT intégrant ECDSA, le partage de secret de Shamir, une topologie réseau TCP multi-threadée robuste, et un dashboard web moderne, nous avons démontré qu'un cluster distribué peut rester résistant aux attaques logiques et physiques tout en maintenant des performances de traitement optimales.